

Automation of indices creation

db team

April 9, 2026

1 Project Overview

This project is based on the PostgreSQL source code obtained from its official repository:

<https://github.com/postgres/postgres> RE-18 stable version

This project aims to enhance database auto-tuning by dynamically monitoring workload patterns. When queries frequently filter on a specific, unindexed column and force a sequential scan, the system automatically generates an index to optimize future execution.

2 System Design and Implementation

The following modifications were implemented in the PostgreSQL source code:

2.1 Sequential Scan Detection

For every incoming query, the system determines whether the execution plan involves a sequential scan. It is then written in logs to indicate the presence of a sequential scan.

2.2 Logging Mechanism

If a query is executed using a sequential scan, relevant metadata is recorded in logs:

- Table name
- Column(s) involved

These logs help track frequently accessed parts of the database.

2.3 Initial implementation :: Local Counter

A local counter is maintained for each (table, column) pair on a per-user basis. Each time a sequential scan occurs:

- The corresponding counter is incremented
- The information is recorded in logs

Initially, a session-local counter was implemented as a proof-of-concept to validate the query tracking logic. Once verified, the architecture was upgraded to a global shared memory counter to support real-world, concurrent database connections.

2.4 Implementation :: Shared Counter

A shared counter is maintained across all users of the database. This counter aggregates sequential scan frequencies for each (table, column) pair.

2.5 Index Recommendation Logic

Based on the shared counter values, the system determines whether index creation may be beneficial using a predefined threshold:

- If the counter exceeds the threshold, it indicates frequent sequential scans
- This suggests that creating an index could improve query performance

3 Experimental Results

To evaluate the effectiveness of the proposed system, we conducted an experiment using two different users executing identical queries on the same table.

3.1 Experimental Setup

- Table: test_table
- Query: `SELECT * FROM test_table WHERE id = 1;`
- User A executed the query 3 times
- User B executed the query 2 times

3.2 Observed Logs

Server Log Output

```
AUTOTUNE (SHARED): Started tracking new scan on Table 16392, Column 1.
(Current count:  1)
AUTOTUNE (SHARED): Incremented counter for Table 16392, Column 1. (Current
count:  2)
AUTOTUNE (SHARED): Incremented counter for Table 16392, Column 1. (Current
count:  3)
AUTOTUNE (SHARED): Incremented counter for Table 16392, Column 1. (Current
count:  4)
AUTOTUNE (SHARED): Incremented counter for Table 16392, Column 1. (Current
count:  5)
AUTOTUNE (SHARED) THRESHOLD CROSSED: Scans for Table 16392, Column 1 reached 5.
Action:  CREATE INDEX.
```